Metro Ridership Forecasting using Inter-Station-Aware Transformer Networks

Big Data Course Presentation (10 minutes max)

Nima DAVARI Maksym DOLHOV Hồ Báo Khánh Nguyễn Mehdi AGHAEI

Big Data Course

Presentation Roadmap

- Problem and Motivation
- ISA-TF Architecture
- Experimental Evaluation
- Key Insights and Discussion

[1/12] Problem Context (Presenter 1)

- Metro systems need station-level inflow/outflow forecasts for operations.
- Accurate forecasts improve:
  - train scheduling
  - staffing
  - maintenance planning
- Poor forecasts $\rightarrow$ congestion and delays
- The task is hard because demand is both **temporal** and **network-dependent**.

[2/12] Research Gap and Paper Idea (Presenter 1)

- Prior methods include statistical, machine learning, and hybrid approaches.
- Many RNN/GCN-based methods are strong for short horizon but can degrade for longer horizon.
- This paper proposes **ISA-TF**: a unified transformer framework for metro ridership forecasting.
- Core idea: fuse ridership history with inter-station topology information.

[3/12] Claimed Contributions (Presenter 1)

- New **Inter-Station-Aware Transformer Networks** architecture.
- Uses three metro graph views: physical, semantic similarity, and correlation.
- Evaluated on SHMetro and HZMetro benchmarks.
- Reports stronger long-horizon performance and a much lower parameter count.

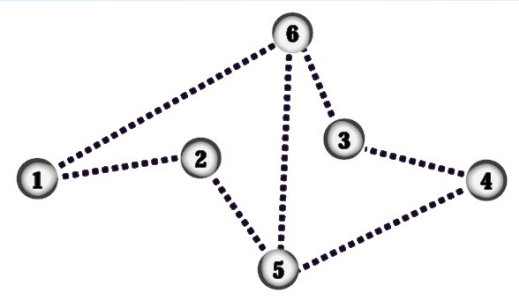
[4/12] Problem Formulation (Presenter 2)

- Input: past ridership sequence and metro graph:

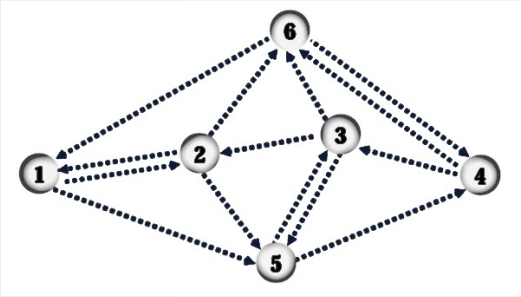
$$\{D^N_{t-n+1}, \ldots, D^N_t\},\ G \rightarrow \{\hat{D}^N_{t+1}, \ldots, \hat{D}^N_{t+m}\}$$

- Each station signal contains two values: inflow and outflow.
- In experiments: $n = 4$ past intervals (60 min total), $m = 4$ future intervals.
- Inference is autoregressive, so horizons longer than 60 min are possible.

[5/12] Metro Graph Representation (Presenter 2)



Physical topology view



Enhanced inter-station relation view

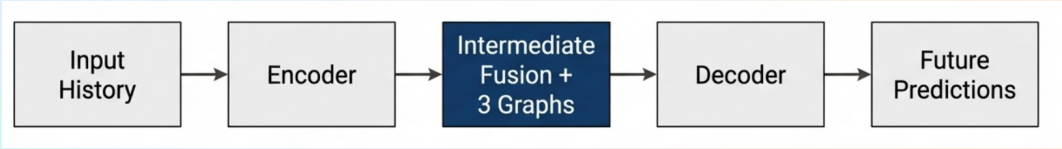
Explanation: The paper represents the metro network with complementary graph structures, not only direct neighboring links. This helps the model capture both local structure and richer cross-station interaction patterns, which improves forecasting robustness when passenger flow dependencies are not purely geographic.

Why Use Transformers?

- RNN-based models struggle with long-range temporal dependencies
- Transformers use **self-attention** to capture long-term patterns
- Parallel computation improves scalability
- ISA-TF integrates network topology into the transformer framework

[6/12] ISA-TF Architecture (Presenter 2)

- Transformer encoder–decoder backbone
- Encoder processes historical ridership
- Graph information fused using:
  - M_p physical topology
  - M_s semantic similarity
  - M_c correlation matrix
- Decoder predicts future ridership autoregressively



ISA-TF Architecture Pipeline

[7/12] Datasets (Presenter 3)

Property	SHMetro	HZMetro
Period	Jul–Sep 2016	Jan 1–25, 2019
Stations	288	80
Physical edges	958	248
Avg daily passengers	8.82M	2.35M
Interval	15 minutes	15 minutes

SHMetro has 811.8M recorded transactions.

[8/12] Experimental Setup (Presenter 3)

- Preprocessing creates the three graph types per dataset.
- Training details: Adam optimizer, 250 epochs, L_2 loss.
- Model hyperparameters: embedding size 512, 8 attention heads.
- Evaluation metrics: RMSE(Root Mean Square Error) and MAE(Mean Absolute Erro).
- Baselines: HM(Historical Mean),
  GB(Gradient Boosting),
  MLP(Multiple Layer Perceptron),
  RNN-GRU(recurrent neural network-Gated Recurrent Unit, it consists of two GRU layers with its hidden units set to 256.),
  STG2Seq(Spatial-temporal Graph to Sequence),
  DCRNN(Diffusion Convolutional Recurrent Neural Network),
  PVCGN(Physical-Virtual Collaboration Graph Network).

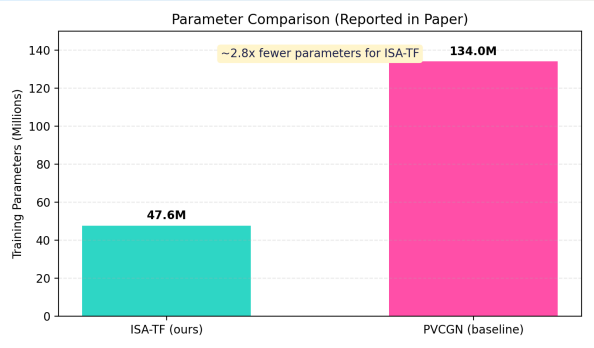
[9/12] Main Results (Presenter 3)

Dataset	Horizon	ISA-TF (RMSE/MAE)	PVCGN (RMSE/MAE)
SHMetro	60 min	**48.43/23.87**	55.27/26.29
HZMetro	60 min	**41.68/24.91**	42.61/24.93
SHMetro	15 min	45.85/**23.05**	**44.97**/23.29

- Strong long-horizon robustness is the key result.
- Short horizon remains competitive with the prior SOTA.

Key Result: ISA-TF performs best on long-horizon prediction (45–60 min).

[10/12] Efficiency Comparison (Presenter 4)



Explanation: Based on the paper's reported values, ISA-TF (Inter-Station-Aware Transformer Networks model) uses 47.6M parameters while PVCGN (Physical-Virtual Collaboration Graph Network) requires about 134M. This means ISA-TF is around 2.8x lighter, which is important for faster training, lower memory use, and easier deployment while keeping competitive or better forecasting performance.

Source: values reported by the paper (Figure 2 discussion and results section).

[11/12] Discussion for Class (Presenter 4)

- **Strengths:** unified architecture, good long-horizon behavior, parameter efficiency.
- **Caveat:** evaluation is on two datasets only; generalization should be tested further.
- **Possible extensions:**
  - Include weather/events/holidays as exogenous features.
  - Add uncertainty intervals, not only point forecasts.
  - Evaluate transfer across cities with different network shapes.

[12/12] Conclusion and Q&A (Presenter 4)

- ISA-TF combines ridership history with topology-aware graph information.
- It matches or beats prior methods, especially for 45–60 minute horizons.
- It achieves this with about 3x fewer parameters than PVCGN.

Takeaway: topology-aware transformers are a strong option for scalable metro ridership forecasting.

Reference: K. Saleh, A.-S. Mihaita, and Y. Ou, 2023 IEEE ITSC (Manuscript 470 preprint).

Thank You

Questions?

1. The Problem: Time-Series Forecasting

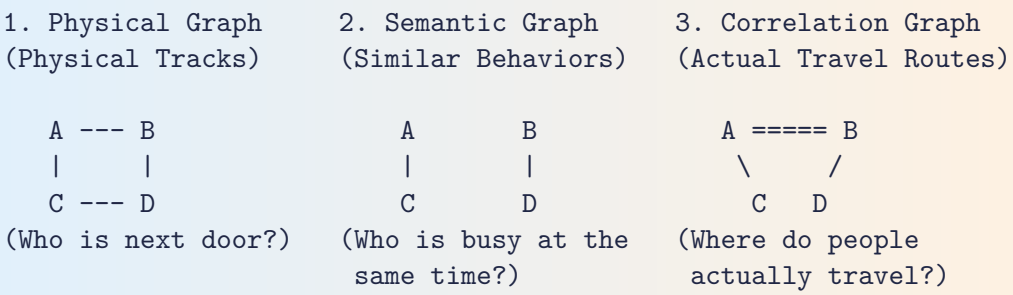
What are we trying to do? We want to predict how many passengers will enter and exit a metro station in the future based on past data

The Notation (The Math):

- We look at a sequence of past data with length n: $(D^N_{t-n+1}, D^N_{t-n+2}, ..., D^N_t)$.
- We want to predict a sequence of future data with length m: $(\hat{D}^N_{t+1}, \hat{D}^N_{t+2}, ..., \hat{D}^N_{t+m})$.
- N is the total number of stations.
- $D \in \mathbb{R}^2$ means the data has two real-number values: inflow count and outflow count.
  Inflow Count: The number of people walking *into* the station and tapping their tickets to get on a train.
  Outflow Count: The number of people getting off the trains, walking *out* of the station, and tapping their tickets to leave into the city.

2. Graph Theory: Mapping the Metro

In Big Data, we represent networks using **Graphs**.This paper uses THREE different graphs to give the AI the full picture:



- **Physical Edge Weight (M_p):** Based on actual track topology.
- **Semantic Similarity (M_s):** Uses Dynamic Time Warping (DTW) for passenger flow patterns.
- **Correlation Matrix (M_c):** Counts total travels from station A to B.

2.1 What is an "Edge Weight"?

In Graph Theory, an **Edge** is a line connecting two nodes (stations). But we don't just want to know IF they are connected; we want to know HOW STRONGLY they are connected.
This strength is called the **Edge Weight**.
The paper calculates edge weights in three completely different ways to give the AI three different "views" of the metro system:

1. Physical (The tracks)
2. Semantic (The passenger behavior)
3. Correlation (The actual travel routes)

2.2 Physical Edge Weight (M_p)

The Concept: Are these stations directly next to each other on the actual train tracks?

- If there is a track between Station A and Station B, the base score is 1 ($E(a, b) = 1$).
- If there is NO track between them, the score is 0 ($E(a, b) = 0$).

The Math (M_p): The formula just turns these 1s and 0s into fractions (normalization).

$$M_p(a, b) = \frac{E(a, b)}{\sum_{k=1}^{N} E(a, k)}$$

Why it matters: If Station A suddenly gets crowded, the physical graph tells the AI that the station right next door is probably going to get crowded a few minutes later.

2.3 Semantic Similarity (M_s) & DTW

The Concept: Do these stations have the same "vibe"? Imagine two business district stations on opposite sides of the city. They are not physically connected, but they BOTH get packed at 5:00 PM.

What is Dynamic Time Warping (DTW)? It is an algorithm that compares two time-series curves to see how similar they are, even if one is slightly delayed.

- If Station A peaks at 5:00 PM and Station B peaks at 5:15 PM, DTW recognizes they have the same behavior pattern and gives them a high similarity score (F_s).

$$M_s(a, b) = \frac{F_s(a, b)}{\sum_{k=1}^{N} F_s(a, k)}$$

Why it matters: It connects stations that share demographic behaviors, allowing the AI to learn broad city-wide patterns.

2.4 Correlation Matrix (M_c)

The Concept: Are people actually riding the train directly from Station A to Station B?

- The physical graph shows the tracks.
- The semantic graph shows the schedule/behavior.
- **The correlation graph shows the actual humans.** It counts the total number of travels that started at Station A and ended at Station B.

The Math (M_c): It takes the number of trips between A and B (R_c) and divides it by the total trips in the whole metro network to get a percentage.

$$M_c(a, b) = \frac{R_c(a, b)}{\sum_{k=1}^{N} R_c(a, k)}$$

Why it matters: If a concert ends at Station A, and historical data shows 80% of concertgoers travel to Transfer Station B, this graph tells the AI to immediately predict a massive inflow at Station B.

3. AI Architectures: RNNs vs. Transformers

The Old Way: RNNs (Recurrent Neural Networks)

- RNNs read data chronologically, step-by-step.
- Examples of sequential data:(Sentences (word by word), Time series (stock prices, weather), Speech signals, Video frames)
- *The Flaw:* They struggle to remember old information. They are challenged when it comes to long-term prediction horizons.

The New Way: Transformers (What this paper uses)

- Transformers don't read step-by-step. They use a mechanism called **Self-Attention**.
- Self-attention works by comparing Query and Key vectors to compute relevance scores, normalizing them with softmax, and using them to weight the Value vectors.
- **Attention** allows the AI to look at ALL past data simultaneously and score which specific time steps are the most important.

3.0 Embeddings: Converting Tokens into Vectors

Problem: Neural networks cannot process raw words or symbols.

- Words, tokens, or time steps must first be converted into numerical representations.
- **Embeddings** transform each token into a dense vector of real numbers.
- Example:

$$\text{cat} = [0.12, -0.44, 0.91, 0.33]$$

$$\text{dog} = [0.10, -0.40, 0.88, 0.30]$$

- Similar tokens obtain similar vectors in the embedding space, so embeddings capture semantic similarity (king - man + woman $\approx$ queen).
- These embeddings become the input used to compute **Query (Q)**, **Key (K)**, and **Value (V)** vectors in the Transformer.

3.1 Self-Attention: Representing Each Token

Step 1: Convert each token into vectors

- Each word (or time step in a time series) is first converted into a numerical vector using embeddings.
- From this vector, the model creates three new vectors using learned weight matrices:
  - **Query (Q)** – what the token is looking for.
  - **Key (K)** – what the token represents.
  - **Value (V)** – the information the token carries.
- These vectors are computed as:

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

- Where X is the input embedding and W_Q, W_K, W_V are learned matrices.

3.2 Computing Attention Scores

Step 2: Measure similarity between tokens

- The model computes how relevant one token is to another.
- This is done using the dot product between the **Query** of one token and the **Key** of another.

$$score(i, j) = Q_i \cdot K_j$$

- In transformers, the scaled version is used:

$$\text{Attention Score} = \frac{QK^T}{\sqrt{d_k}}$$

- The scaling factor $\sqrt{d_k}$ stabilizes training when the vector dimension becomes large.
- Higher scores mean the model should pay more attention to that token.

3.3 Attention Weights and Final Output

Step 3: Convert scores into attention weights

- The raw attention scores are converted into probabilities using the **Softmax** function:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- The softmax ensures all attention weights sum to 1.
- These weights determine how much each token contributes to the final representation.
- The model then combines the **Value vectors** according to these weights to produce the final output.
- This allows the transformer to focus on the most relevant parts of the sequence.

4. The Paper's Invention: ISA-TF

The **Inter-Station-Aware Transformer Networks (ISA-TF)** combines the time-reading power of Transformers with the spatial map of the 3 Graphs.

```
[PAST DATA] --> (ENCODER) --> [FUSION] --> (DECODER) --> [FUTURE DATA]
Last 60 mins   Finds Time   Mixes Time     Guesses the   Next 60 mins
               Patterns     with 3 Graphs    Future
```

- **Encoder:** Uses self-attention to capture interactions in the historical sequence.
- **Intermediate Fusion:** Fuses the encoder output with the three network graphs via concatenation.
- **Decoder:** Attends to both the encoder and the fused graph data to auto-regressively predict the future.

4.1 ISA-TF Architecture Overview

ISA-TF (Inter-Station-Aware Transformer Networks)

- The model combines **temporal patterns** and **spatial relationships** between stations.
- Temporal dependencies are captured using a **Transformer architecture**.
- Spatial relationships between stations are represented using **three graph structures**.
- Overall pipeline:

$$\text{Past Data} \rightarrow \text{Encoder} \rightarrow \text{Graph Fusion} \rightarrow \text{Decoder} \rightarrow \text{Future Prediction}$$

- Goal: predict the **next 60 minutes** using the **previous 60 minutes of observations**.

4.2 ISA-TF Components

Encoder

- Uses Transformer self-attention.
- Captures temporal patterns in the historical sequence.
- Produces a representation summarizing the past observations.

Intermediate Fusion

- Combines the encoder output with three station graphs.
- Graphs represent spatial relationships between stations.
- Fusion is performed using **concatenation**.

Decoder

- Uses attention to both the encoder output and fused graph data.
- Predicts future values **auto-regressively**.
- Generates the future sequence step-by-step.

4.3 Encoder: Learning Temporal Patterns

Input:

- Historical observations from the previous 60 minutes.
- Each time step is converted into an embedding vector.

Processing Steps:

- Positional encoding is added to preserve the order of time.
- Self-attention allows each time step to interact with all other time steps.
- A feed-forward neural network refines the learned representation.

Output:

- A temporal representation summarizing patterns in the historical sequence.

Self-Attention Mechanism

Goal: Allow each time step in a sequence to incorporate information from all other time steps.

Computation:

$$\text{Attention}(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q = Query matrix
- K = Key matrix
- V = Value matrix
- d_k = dimension of key vectors (used for scaling)

Interpretation:

- The dot product QK^T measures similarity between time steps.
- The softmax function converts similarities into attention weights.
- Each time step is represented as a weighted combination of all other time steps.

Feed-Forward Neural Network

Goal: Transform and refine the representations learned by the attention layer.

Computation:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- W_1, W_2 are learned weight matrices
- b_1, b_2 are bias vectors
- $\max(0, \cdot)$ represents the ReLU activation function

Interpretation:

- The first linear transformation expands the feature space.
- The nonlinear activation introduces model expressiveness.
- The second transformation projects features back to the model dimension.

Activation Functions in Neural Networks

Motivation

- Neural networks apply linear transformations to the input data.
- If only linear operations were used, the model would behave like a simple linear model.
- This would limit the ability of the network to learn complex patterns.

Solution: Activation Functions

- Activation functions introduce **nonlinearity** into the network.
- This allows neural networks to approximate complex and nonlinear relationships in data.

ReLU Activation Function

Definition

$$ReLU(x) = \max(0, x)$$

- If $x > 0$, the output equals x.
- If $x \leq 0$, the output is 0.

Properties

- Simple and computationally efficient.
- Introduces nonlinearity into the model.
- Helps mitigate the vanishing gradient problem.

Usage in Transformers

- Applied inside the feed-forward neural network after the first linear transformation.

4.4 Intermediate Fusion: Adding Spatial Information

Goal:

- Combine temporal patterns from the encoder with spatial relationships between stations.

Spatial Information:

- The model uses three graphs describing relationships between stations.
- These graphs encode spatial dependencies such as distance or mobility flows.

Fusion Strategy:

- The encoder output is merged with graph features.
- The paper performs this fusion using **concatenation**.

4.5 Concatenation in Feature Fusion

Concatenation:

- Concatenation means joining two vectors into one larger vector.

Example:

$$\text{Temporal Features} = [0.3, 0.5, 0.2]$$

$$\text{Graph Features} = [0.7, 0.1]$$

$$\text{Concatenated Vector} = [0.3, 0.5, 0.2, 0.7, 0.1]$$

- This allows the model to use both temporal and spatial information simultaneously.

4.6 Decoder: Predicting Future Values

Role of the Decoder:

- Generate predictions for the next 60 minutes.

Processing Steps:

- Masked self-attention ensures the model only sees past predictions.
- Cross-attention allows the decoder to use the encoder output and fused graph information.
- A feed-forward network refines the predictions.

Prediction Strategy:

- The model predicts future values **auto-regressively**, one step at a time.

5. The Math: Evaluation Metrics

To prove their AI works, researchers test it on historical data (where they already know the answers) and measure the errors. Lower scores are better.

RMSE (Root Mean Square Error): Heavily punishes large errors because the differences are squared before being averaged.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{D}_i - D_i)^2}$$

MAE (Mean Absolute Error): The simple average of how far off the AI's guesses were.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|\hat{D}_i - D_i|$$

Note: $\hat{D}_i$ is the predicted ridership, and D_i is the actual ground-truth ridership.

6. What is Time-Series Data?

Time-Series is just data collected over time, in order.
The paper frames ridership prediction as a time-series forecasting problem. Imagine counting people at a door every 15 minutes.

- t represents the *current* time interval (e.g., 10:00 AM).
- $t-1$ is the *previous* interval (9:45 AM).
- $t+1$ is the *next* interval (10:15 AM).

When the paper says it uses $n = 4$ historical steps to predict $m = 4$ future steps, it means: *"I will look at the last 60 minutes (four 15-min chunks) to guess the next 60 minutes."*

7. How Does the AI Actually "Learn"?

Machine learning models aren't magic; they use trial and error.

1. **Prediction:** The AI makes a guess about the future.
2. **Loss Function:** We measure how wrong the guess was. The paper uses the **L2-loss function**, which heavily penalizes big mistakes.
3. **Optimizer:** A mathematical tool that tweaks the AI's internal math to make the next guess better. The paper uses the **Adam optimizer**.
4. **Epochs:** One full cycle of training through all the data. This model trained for **250 epochs** (it practiced 250 times).

8. Unpacking "Self-Attention" (The Transformer's Secret)

The core of a Transformer network is the **Self-Attention** layer.
Analogy: Reading a sentence. If you read: "The **bank** of the **river** was muddy."
To understand the word "bank", your brain pays *attention* to the word "river" (not a money bank).
In the metro paper, to predict a crowd at 5:00 PM, the Self-Attention mechanism looks at the sequence of past data and learns to assign higher "attention scores" to 4:45 PM and 4:30 PM, realizing those specific past moments are the most relevant clues.

9. Decoding the "Baselines" (The Competitors)

The paper compares its new ISA-TF model against older models (baselines) to prove it is better. Here is what they are in simple terms:

- **HM (Historical Mean):** The simplest method. It just calculates the average of past ridership to guess the future.
- **GB (Gradient Boosting):** A classic machine learning method that combines many "weak" decision trees to make a strong prediction.
- **MLP (Multi-Layer Perceptron):** A standard, basic artificial neural network.
- **RNN-GRU:** A recurrent network that processes time step-by-step, but suffers from "forgetting" long-term trends.

10. Why do "Parameters" Matter?

In the results, the paper brags that their model is roughly 3 times more efficient. They prove this by counting **Parameters**.

- Think of parameters as the "gears" or "dials" inside the AI's mathematical engine.
- The previous best model (PVCGN) needed **134.14 Million** parameters to work.
- This new ISA-TF model only needs **47.66 Million** parameters.

Why is this good? A model with fewer parameters trains faster, requires less computer memory, and is easier for a city to actually install and run on their servers!

11. How AI Learns: The "Mountain" Analogy

Before we define Adam or Epochs, we need to understand the AI's goal.
Imagine the AI is standing at the top of a mountain, blindfolded.

- The **top of the mountain** represents being 100% wrong (high error/loss).
- The **bottom of the valley** represents being 100% right (zero error).

The AI wants to walk down to the bottom.

- The **Loss Function** (like the L2-loss in the paper) tells the AI how high up the mountain it currently is.
- The **Optimizer** is the strategy the AI uses to feel the ground with its foot and decide which direction to step next to go downhill.

12. What is the Adam Optimizer?

An **Optimizer** is the mathematical engine that updates the AI's internal numbers (parameters) after it makes a guess.
Adam stands for *Adaptive Moment Estimation*. It is currently one of the most popular and powerful optimizers in the world.

- Instead of taking the same size step every time, Adam is *smart*.
- If the mountain is steep, Adam takes big, fast steps.
- If the AI feels it is getting close to the bottom (the right answer), Adam automatically takes tiny, careful steps so it doesn't overshoot the target.

13. What is an Epoch?

The paper mentions the training "lasted for 250 epochs".
Think of the training data (the historical metro ridership) as a giant stack of flashcards.

- An **Epoch** is exactly ONE full pass through the entire stack of flashcards.
- If the AI only looks at the flashcards once (1 epoch), it won't remember much. It will still make bad predictions.
- To get smart, the AI needs to see the same data over and over again.
- By running **250 epochs**, the authors forced the AI to review the entire Shanghai and Hangzhou metro histories 250 times until it had memorized the underlying patterns perfectly!

14. Putting the Training Loop Together

Here is the exact cycle of how the ISA-TF model was trained:

1. **Guess:** The AI looks at 60 mins of past metro data and guesses the next 60 mins.
2. **Check Error (Loss):** It compares its guess to the real historical answer using the L2-loss function.
3. **Learn (Adam):** The Adam optimizer looks at the error and tweaks the AI's internal math (parameters) so it will guess better next time.
4. **Repeat (Epochs):** The AI repeats this process for every single piece of data in the dataset. Once it finishes the whole dataset, that is 1 Epoch. It did this 250 times!

A.1 What is Convolution?

Definition: Convolution is a mathematical operation that extracts local patterns from data by sliding a small filter (kernel) over the input.

- Commonly used in **images, signals, and time series**.
- Allows neural networks to detect features like:
  - Edges or textures in images
  - Spikes or trends in time series

A.2 Convolution Formula

For a 1D signal:

$$y(t) = \sum_{i=0}^{k-1} x(t+i) \cdot w(i)$$

- $x(t)$ = input signal
- $w(i)$ = filter (kernel)
- k = kernel size
- $y(t)$ = output at position t

A.3 Convolution Example

Input signal: $[2, 4, 6, 8]$
Kernel: $[1, 0]$
Sliding window computation:

$$(2 \cdot 1 + 4 \cdot 0) = 2$$

$$(4 \cdot 1 + 6 \cdot 0) = 4$$

$$(6 \cdot 1 + 8 \cdot 0) = 6$$

Output: $[2, 4, 6]$

A.4 Convolution Intuition

- Convolution can be seen as a “pattern detector” that scans the data.
- The kernel highlights where a certain feature occurs.
- Advantages:
  - Detects local patterns efficiently
  - Reduces parameters compared to fully connected layers
  - Works well with structured data like images or time series